

OVERLAP WITH ENVHARNESS

ENVHARNESS VS PROJET COMPLEXE (CANVAS TRANSCRIPTION + DIAGRAMS)

Date: 2026-08-24
Status: architecture note / design instrument (not a spec, not an implementation plan)
Canvas source: Cursor canvas *EnvHarness vs Projet Complexe* (this file includes every word of that canvas)
Reads: [google-research/envharness](#), [arXiv:2608.19880](#), ASC README, August 2026 ideas notes (Revival v2–v4, Reverse Prompting vs CLR, Meadows leverage, Four layers, Cognitive institutions)
This note exists so the canvas comparison can live in the ideas shelf as ordinary markdown, with Mermaid diagrams where a table or a callout is not enough. The prose blocks labeled **Canvas** below are a complete transcription. Nothing from the canvas is omitted. Diagrams and section chrome are additions around that text, not replacements.

CANVAS

ENVHARNESS VS PROJET COMPLEXE

Overlap review of [google-research/envharness](#) and [arXiv:2608.19880](#) against ASC README + August 2026 ideas notes (Revival v2–v4, Flow / CLR, Meadows leverage, four layers). Paper dated 20 Aug 2026.

Verdict

Strong overlap on one axis, weak on the rest. EnvHarness is the closest published operationalization of “keep the agent in the flow band by reshaping the world, not the weights.” It does that for frozen training benchmarks. Projet Complexe wants the same control idea for a living second brain, with ASC as generic glue. Steal the interface taxonomy and the difficulty-band loop. Do not import their gym stack, skill-induction pipeline, or Python Rules-as-source-of-truth.

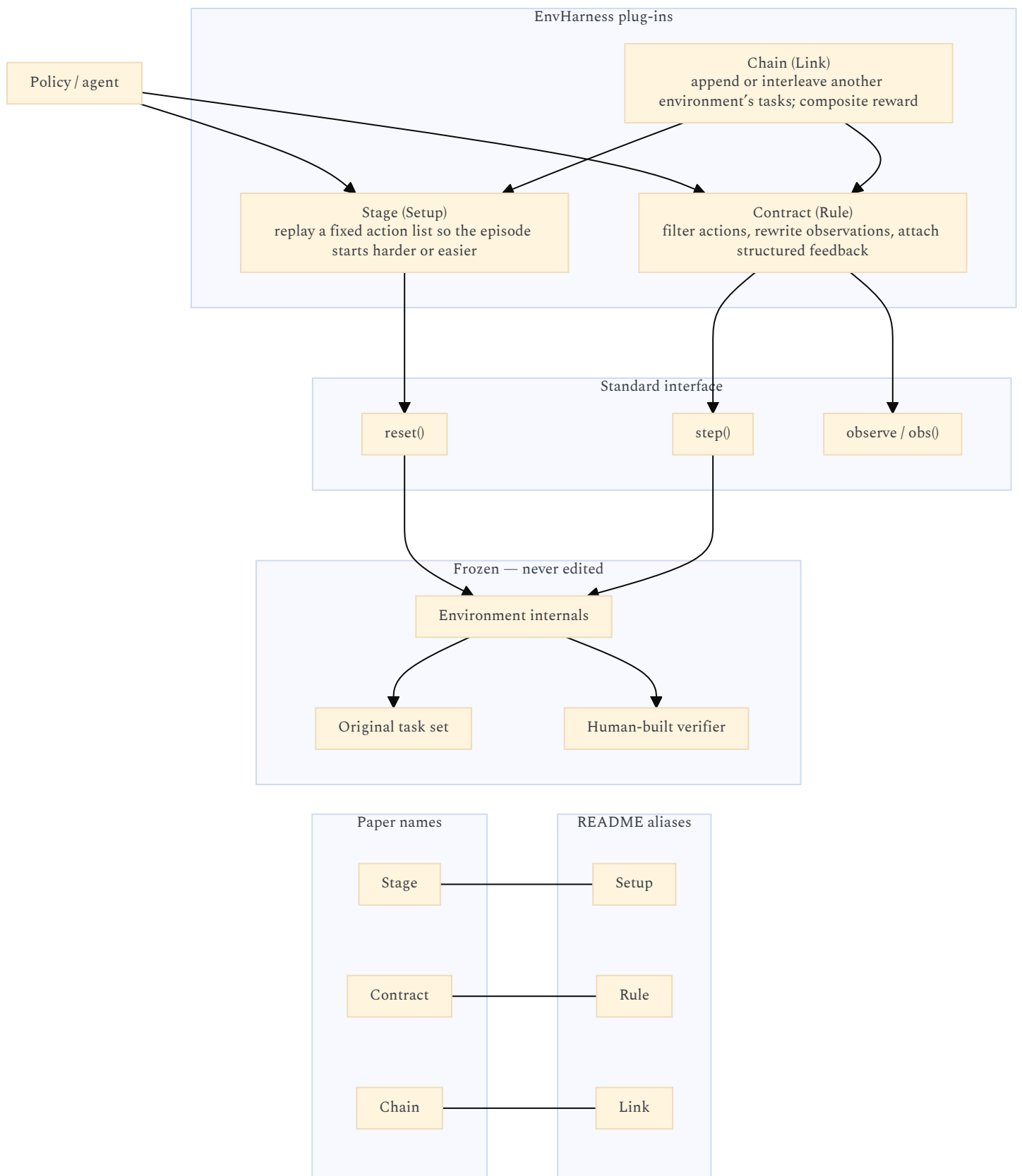
Dimension	Strength	Overlap	EnvHarness	Projet Complexe
Wrap, don’t rebuild	High	Strong overlap — one axis	frozen training benchmarks	living second brain (ASC as generic glue)
Difficulty / flow band	High	Strong overlap — one axis	reshape the env for a flow band	regulate CLR / packing for a flow band
Plug-in composition	Med	Partial	Stage / Contract / Chain	hooks, pivots, ASC compositions
Product and ontology	Low	Weak on the rest	benchmark training stack	ingest, claims, exports, fr/en/pt

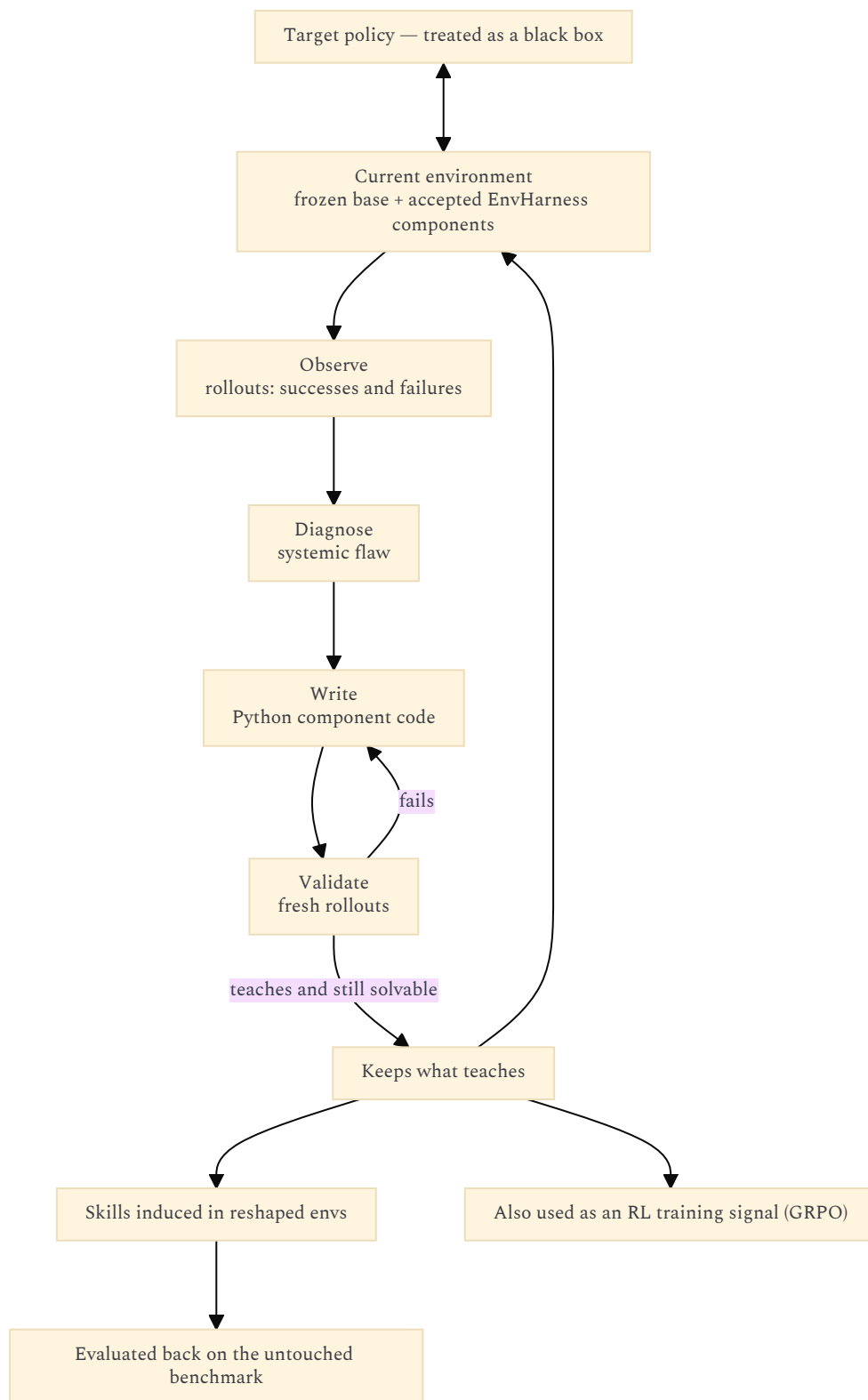
WHAT ENVHARNESS ACTUALLY IS

LLM agents learn from interactive environments, but those environments are expensive and static: they ignore a given policy’s weaknesses and stop teaching once the tasks are solved. Generating new environments is domain-specific and needs new (often unreliable) verifiers. EnvHarness wraps a frozen environment at `reset()` / `step()` only. Three plug-ins (paper names; README aliases in parentheses):

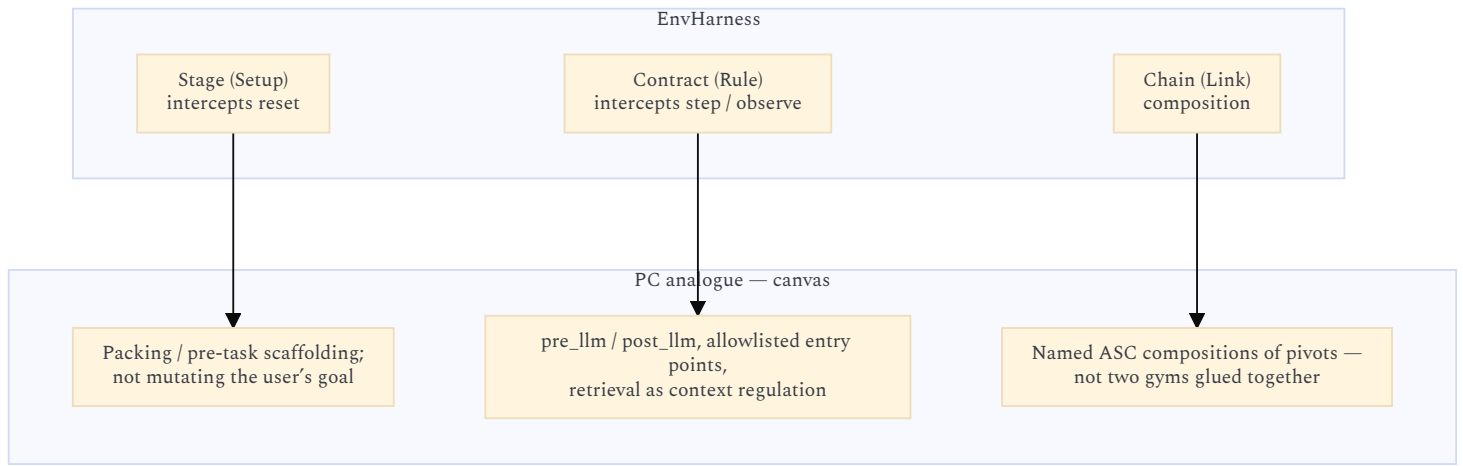
Component	Intercepts	Effect	PC analogue
Stage (Setup)	reset	Replay a fixed action list so the episode starts harder or easier	Packing / pre-task scaffolding; not mutating the user’s goal
Contract (Rule)	step / observe	Filter actions, rewrite observations, attach structured feedback	pre_llm / post_llm, allowlisted entry points, retrieval as context regulation
Chain (Link)	composition	Append or interleave another environment’s tasks; composite reward	Named ASC compositions of pivots — not two gyms glued together

The original task set and human-built verifier stay frozen. EnvRigger (designer agent) observes rollouts, diagnoses a systemic flaw, writes Python component code, validates on fresh rollouts, keeps what teaches. Skills induced in reshaped envs are evaluated back on the untouched benchmark. Gains: up to +9.0 on held-out ALFWorld OOD; ~9.8% fewer steps; also used as an RL training signal (GRPO).



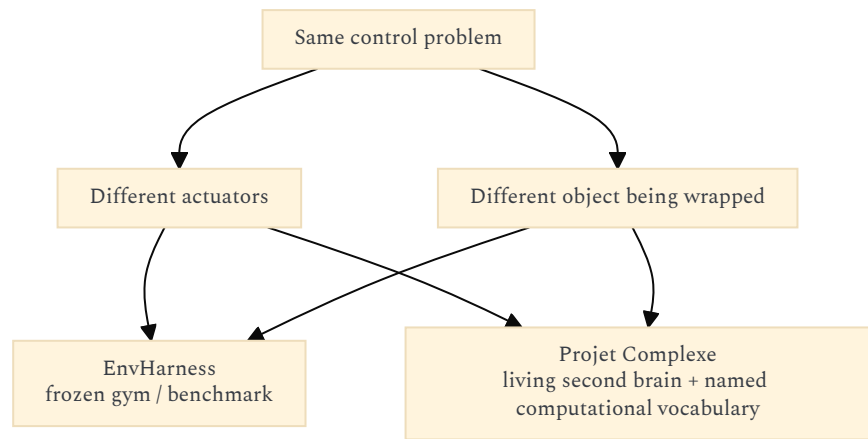


Stage, Contract, and Chain against the three Projet Complexe analogues from the canvas table:



CONCEPT MAP

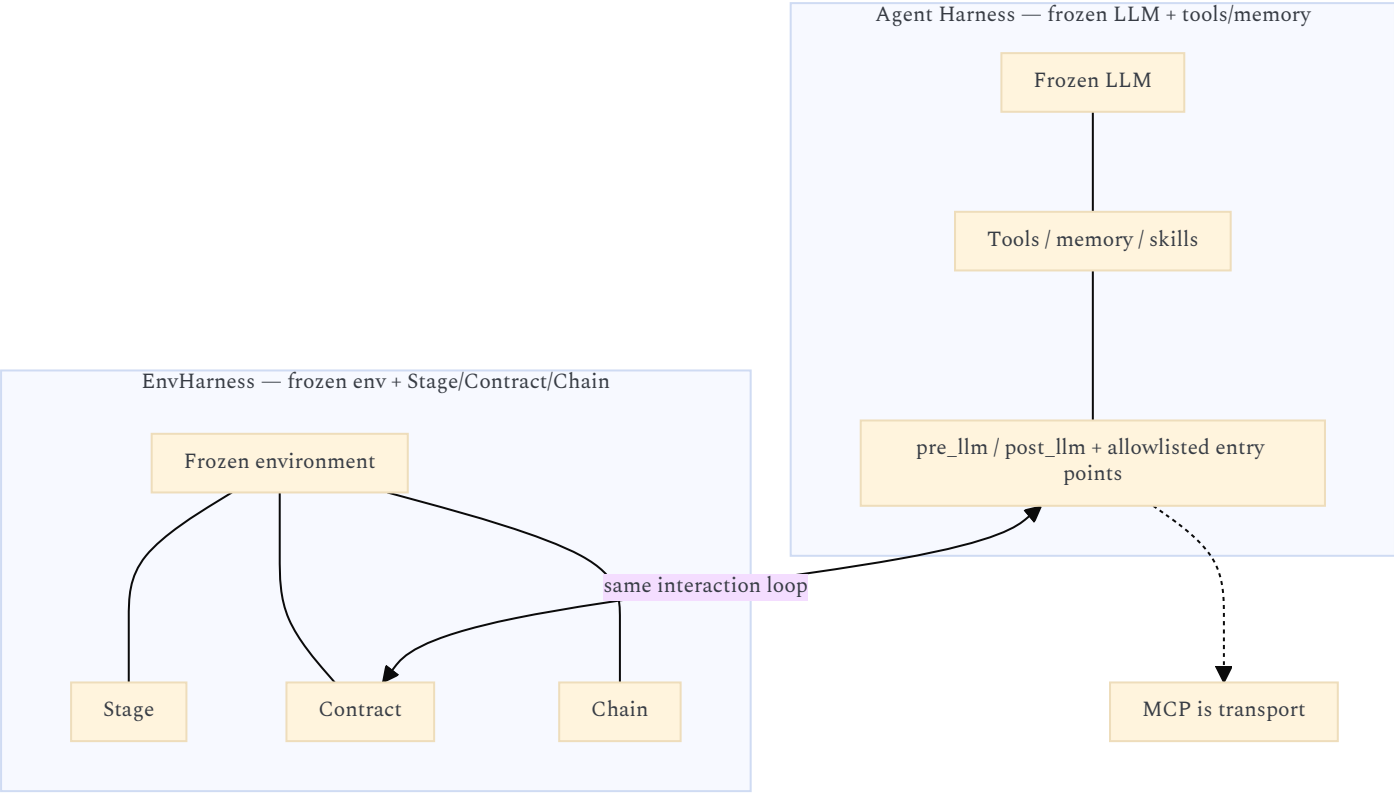
Same control problem, different actuators and different object being wrapped.



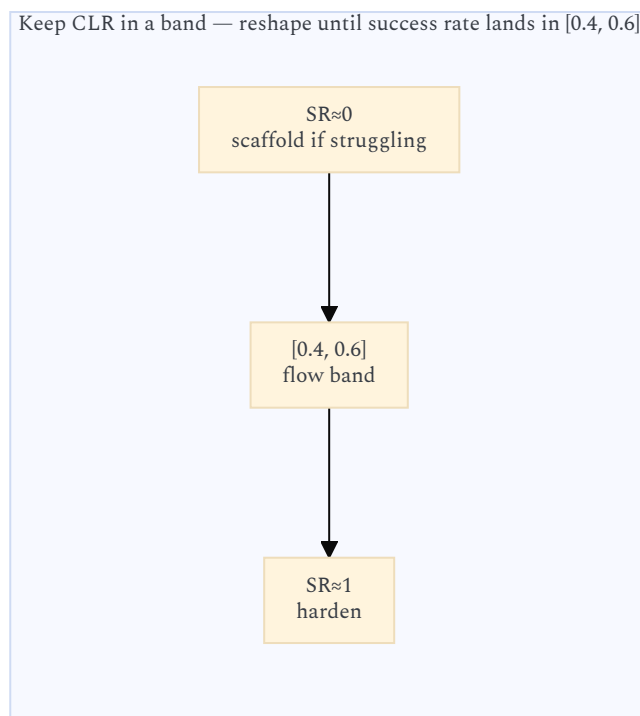
Idea in your notes	EnvHarness counterpart	Overlap
Flow / CLR: task complexity $\approx$ effective capacity; keep CLR in a band	Difficulty zone: reshape until success rate lands in [0.4, 0.6]; scaffold if struggling, harden if SR $\approx$ 1	Highest. This is the same cybernetic claim.
Prompt engineering is challenge regulation, not “being clear”	Contract rewrites obs/actions so the policy cannot take the shortcut it already knows	High. Different lever: they mutate the world; you mutate packing and task shape.
ASC wraps frozen CLIs/OS ops; names persist when implementations swap	ActionableEnv bridge wraps docker/browser/sim; harness layers stack; internals never edited	High as a design pattern. ASC is more general (shell, files, hosts), not gym-only.
v4: agent harness = pre_llm / post_llm + allowlisted entry points; MCP is transport	Paper Figure 2: Agent Harness (frozen LLM + tools/memory) vs EnvHarness (frozen env + Stage/Contract/Chain)	High as a citation. You already decided the agent-side. They publish the env-side.
v3: RAG is context regulation so a 7B stays in the Flow band	Observation rewrite / truncation; hide objects; block high-level teleports	Medium. Same intent (fit the window). They filter a sim; you pack a corpus.
Meadows #6 information flows; #5 rules; #8 balancing feedback	Who sees what (obs maps); which actions are legal (action maps); write-and-validate loop	Medium-high. EnvRigger is an automated #8 loop over the env, not over knowledge commits.
Level 4 in the Flow note: meta-cognitive regulation of the problem before reasoning	EnvRigger Observe $\rightarrow$ Diagnose $\rightarrow$ Write $\rightarrow$ Validate, conditioned on this policy and this task	High as a loop shape. They emit Python wrappers. You want YAML + DSL as source of truth.
Three-project cut: ASC names; PCA composes; PC interprets. Killswitch. HITL claims.	One Python framework: bridges + harness + designer + skill bank + optional RL	Low. Different product, ethics, and ontology.
Knowledge plane: Claims, Links, gaps, provenance, fr/en/pt, modest hardware routing	No knowledge ontology. Benchmarks: ALFWorld, WebArena, SWE-bench, OfficeQA, SpreadsheetBench	None. Do not collapse these.

Idea in your notes	EnvHarness counterpart	Overlap
DSL is filename-safe addressing; JSON Schema is a projection; never teach the model a private DSL as native tool language (v4 + DSL note)	Designer emits real Python subclasses, compiled in an isolated subprocess	Conflict if copied. Their “code as the harness” fights your “YAML/DSL as SoR”.

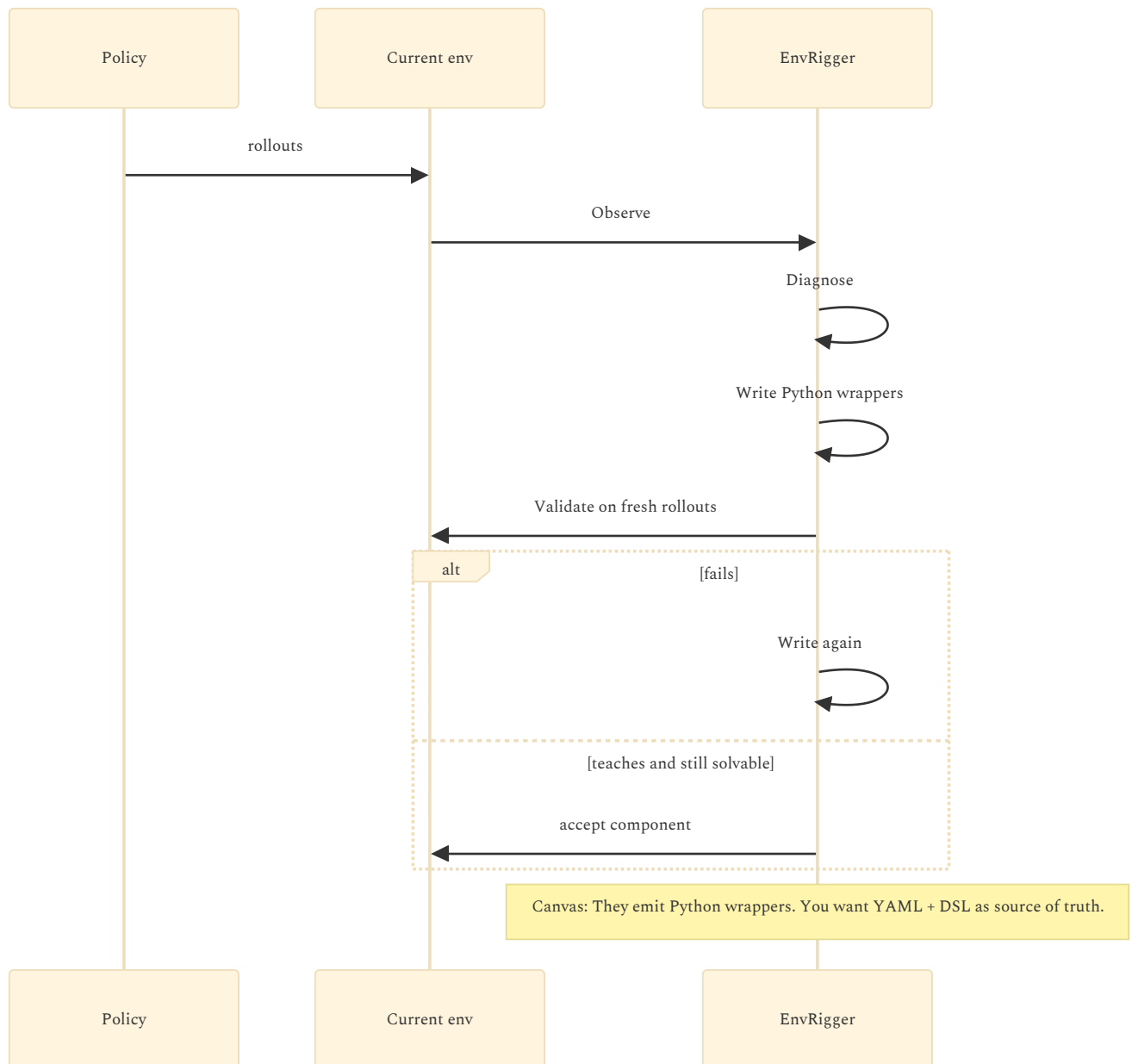
Paper Figure 2 as named in the canvas (agent harness vs environment harness):



Difficulty zone vs Flow / CLR band (the canvas’s highest overlap):



EnvRigger loop shape vs YAML + DSL as source of truth:



STEAL — USEFUL TO PROJET COMPLEXE

Cite EnvHarness as the env-side of the harness idea your v4 note already split from MCP / skills / tools.

Name three actuators explicitly: start-state (Stage), interaction (Contract), composition (Chain). Map them onto packing, allowlists, and ASC compositions — not onto a second gym runtime.

Operationalize CLR as a band, not a minimum. Their result that $SR \approx 1$ and $SR \approx 0$ are equally useless is your boredom vs instability split, measured.

Keep the verifier / human goal frozen. Reshape what the agent sees and may do. Do not silently rewrite the user’s task.

If you ever grow an eval harness for agents, their observe–diagnose–write–validate loop is a better teacher than “generate more tasks.”

REFUSE — NOT YOUR STACK

Do not put EnvRigger, skill banks, or GRPO in ASC core. ASC README already parks complex agent work in dedicated instances.

Do not make Python Rules the source of truth. v4 already chose YAML entity/able → generated JSON Schema / optional MCP adapter.

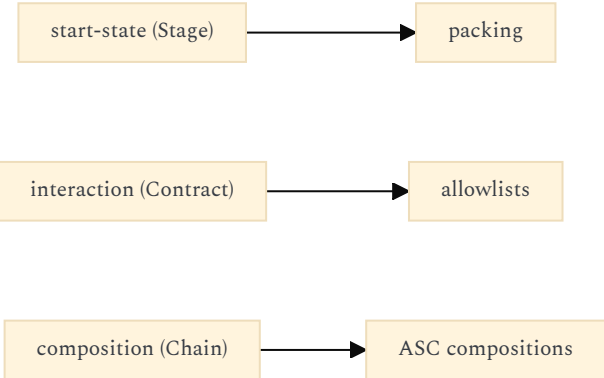
Do not treat ALFWorld / SWE-bench transfer as Proj et Complexe’s success metric. Yours is ingest, curate, export, research, same named actions for human and agent, three languages, modest hardware.

Do not rebuild Pi / Cursor / a coding-agent loop because they wrapped environments. Wrap a harness; do not become one.

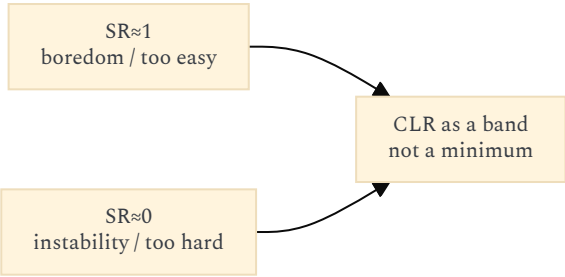
Chain is hard even for them to automate (they excluded it from the designer loop). Do not start with cross-environment episode glue.



Three actuators mapped as the canvas asks (packing, allowlists, ASC compositions — not a second gym runtime):



Boredom vs instability, measured:



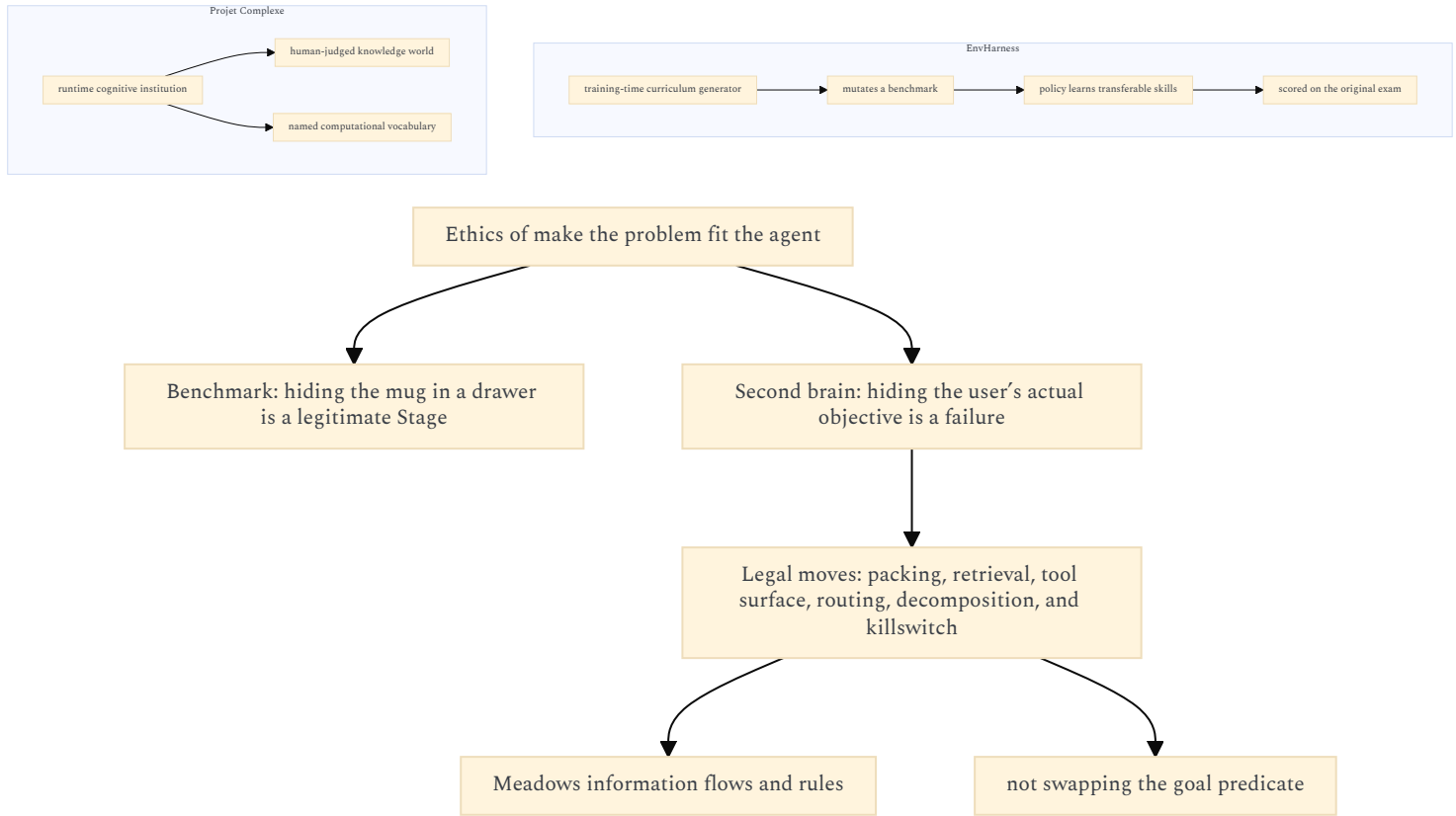
WHERE THE ANALOGY BREAKS

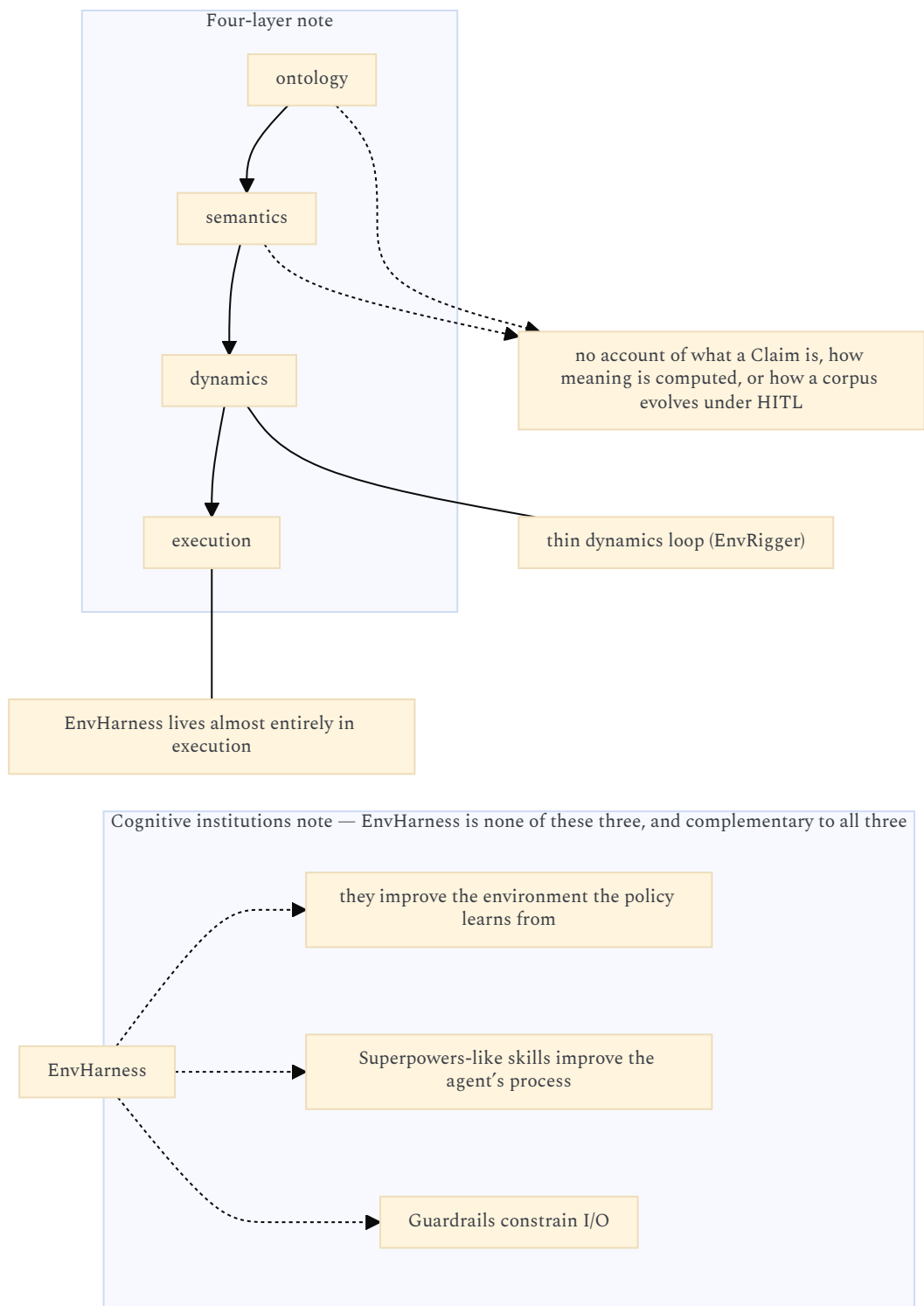
EnvHarness is a **training-time curriculum generator** that mutates a benchmark so a policy learns transferable skills, then is scored on the original exam.

Projet Complexe is a **runtime cognitive institution**: a human-judged knowledge world plus a named computational vocabulary.

That changes the ethics of “make the problem fit the agent.” In a benchmark, hiding the mug in a drawer is a legitimate Stage. In a second brain, hiding the user’s actual objective is a failure. The legal moves on your side are packing, retrieval, tool surface, routing, decomposition, and killswitch — Meadows information flows and rules — not swapping the goal predicate.

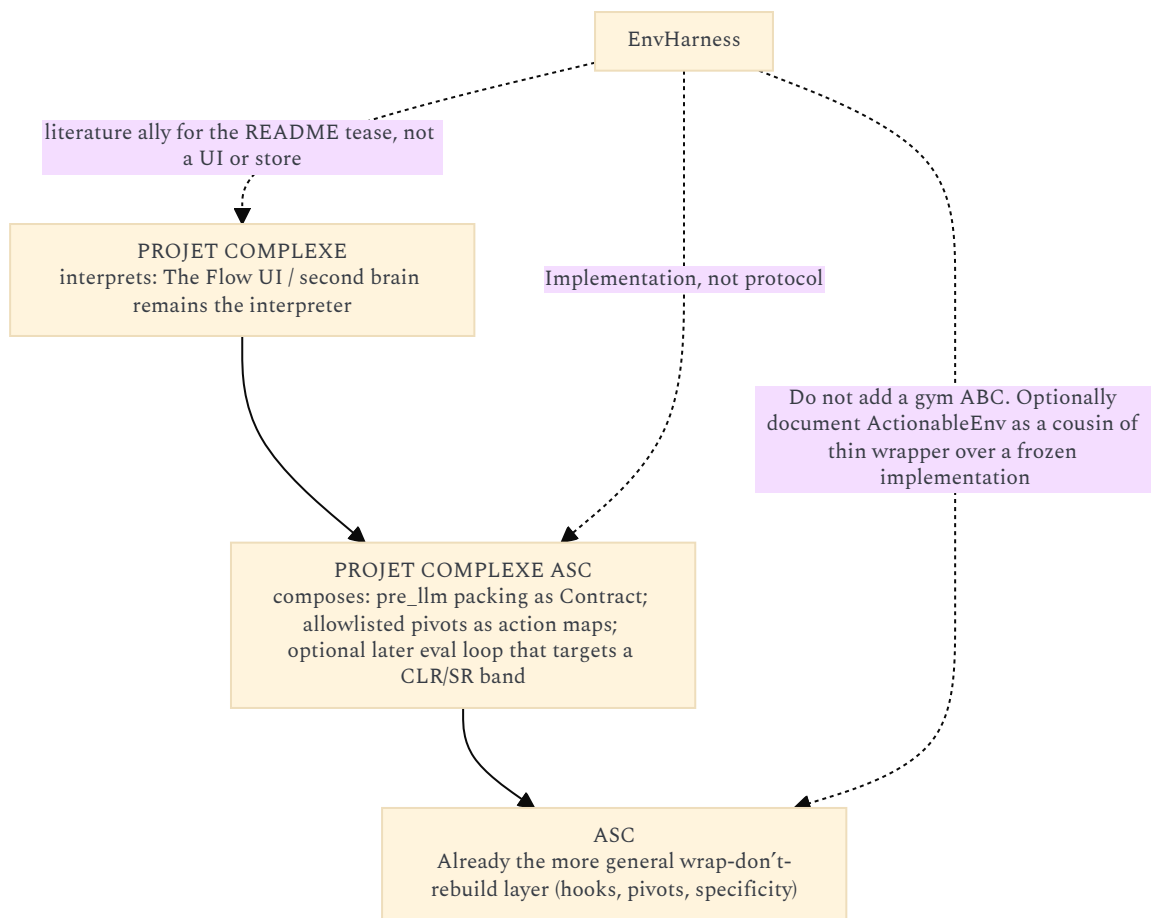
Four-layer note (ontology / semantics / dynamics / execution): EnvHarness lives almost entirely in execution, with a thin dynamics loop (EnvRigger). It has no account of what a Claim is, how meaning is computed, or how a corpus evolves under HITL. Cognitive institutions note: they improve the environment the policy learns from; Superpowers-like skills improve the agent’s process; Guardrails constrain I/O. EnvHarness is none of those three, and complementary to all three.





PLACEMENT ON YOUR THREE-PROJECT CUT

Project	Relation to EnvHarness
ASC	Already the more general wrap-don't-rebuild layer (hooks, pivots, specificity). Do not add a gym ABC. Optionally document ActionableEnv as a cousin of "thin wrapper over a frozen implementation."
Projet Complexe ASC	If anything is stolen, it is policy: pre_llm packing as Contract; allowlisted pivots as action maps; optional later eval loop that targets a CLR/SR band. Implementation, not protocol.
Projet Complexe	The Flow UI / second brain remains the interpreter. EnvHarness does not help ingest, claims, exports, or fr/en/pt. It is a literature ally for the README tease, not a UI or store.



Sources: EnvHarness README (Setup / Rule / Link, ActionableEnv, designer loop, skill tables); paper abstract + §§1-3, 5 (Stage / Contract / Chain, EnvRigger, difficulty-band experiment, related work); ASC README (wrap, Flow tease, agent extension, DSL); Revival v2-v4; Reverse Prompting vs CLR; Meadows leverage; Four layers; Cognitive institutions.

- envharness.com
- [projet-complexe](https://projet-complexe.com)
- [projet-complexe-asc](https://projet-complexe-asc.com)